Reliable Federated TinyML Deployment for IoT SecurityReliable Federated TinyML for IoT Security

Reliable Federated TinyML Deployment for IoT Security

Anonymous Author(s)

Abstract

Federated learning (FL) and TinyML together promise privacy-preserving intrusion detection on microcontrollers, but whether a federated model can be compressed for such devices without pooled data, and without losing detection quality, is rarely tested end to end. We train an MLP intrusion detector with Flower on CIC-IDS2017 and TON_IoT and compress it for TensorFlow Lite Micro using only data that participating clients hold. Federated training comes within 1.7 F1 points of an identically configured centralized baseline and loses less than one F1 point under a Dirichlet(0.3) partition; a local-only baseline shows that federation matters most for clients with few local attacks, which detect 45–93% of attacks alone and 99.8–99.99% with the federated model. Structured pruning followed by fine-tuning and QAT fine-tuning on one client's data yields a 65 KB INT8 model (12.3×) that matches the float federated model. For recall-priority deployment, distillation from the federated model, fine-tuning on clipped inputs across clients, and a threshold chosen on client-held data let a 65 KB model miss 27 of 85,173 CIC-IDS2017 test attacks (FP32 model at its default threshold: 52) at an 11.3% false-alarm rate, and a 57 KB TON_IoT model miss 12 of 16,215 at 6.6%. On an ESP32 the deployed INT8 models classify a flow in 26 ms (CIC-IDS2017, 112 KB) and 13 ms (TON_IoT), 4.4–7.9× faster than their FP32 versions, with all on-device decisions matching the host. We also find that training-time QAT inside FL collapses on heavy-tailed CIC-IDS2017 features, that full-integer post-training quantization is unstable across calibration draws until the calibration inputs are clipped, and that four silent failures in a common TensorFlow/tfmot/Flower toolchain make wrong compressed-model results look plausible; we give the checks that catch them.

Keywords: federated learning, TinyML, intrusion detection, quantization, pruning, microcontrollers

1 Introduction

The number of Internet of Things (IoT) devices in homes, healthcare and industrial control keeps growing, and so does their exposure to network intrusions [1]. Machine-learning intrusion detection systems (IDS) detect such traffic well [46], but they are usually trained centrally on pooled traffic, which raises privacy and ownership concerns. Federated learning (FL) lets clients train locally and share only model updates [27, 36], and TinyML compresses models until they run on microcontrollers [3, 10]. Combining the two raises a question that is easy to state and, as we found, easy to get wrong: can a model trained without pooling data be compressed to microcontroller size without losing what federated training learned, and without quietly using pooled data during compression?

Intrusion detection is a demanding test case. A missed attack usually costs more than a false alarm, benchmark datasets are imbalanced, and their standardized features are heavy-tailed. We study this combination on two datasets with an MLP detector and make five contributions:

- An end-to-end evaluation of federated training against an identically configured centralized baseline under near-IID and Dirichlet(0.3) partitions, together with a local-only baseline that locates the benefit of federation in clients with few local attacks (§4.1).
- An FL-faithful compression procedure (exact BatchNorm folding, structured pruning, fine-tuning and QAT fine-tuning on one client's data) and its decomposition, compression-strength sweep, and dependence on the fine-tuning client (§4.2).
- A comparison of five post-training quantization methods and 16 complete pipelines combining structured and unstructured pruning, PTQ, QAT and distillation, including the instability of full-integer PTQ across calibration draws and a one-line fix (§4.3,§4.7), and a comparison with calibration estimators, PTQ-initialized and learned-clipping QAT, and server-side data-free calibration from the literature, which locates the problem in the input quantization range (§4.4).
- A recall-priority deployment recipe that uses only client-held data and lets compressed models miss fewer attacks than the FP32 model at its default threshold (§4.8).
- A negative result for training-time QAT inside FL on heavy-tailed features, and four silent toolchain failure modes with the invariant check that exposes them (§4.5,§5).

The paper is an evidence-backed characterization rather than a new algorithm. All numbers come from our own runs; the table captions name the run that produced them (Appendix D).

2 Background and Related Work

Intrusion detection datasets. CIC-IDS2017 [46] is the most widely used flow-based IDS benchmark; Engelen et al. [13] document labelling and flow-construction errors in

it, which makes absolute numbers hard to compare across papers. TON_IoT [38] and Bot-IoT [28] target IoT traffic. We use CIC-IDS2017 and the TON_IoT network dataset.

Federated learning. FedAvg [36] averages client updates; FedAvgM [22], which we use, adds server momentum and is evaluated there on Dirichlet label partitions, the non-IID model we adopt. Zhao et al. [50] characterize the degradation under skewed client data, and FedProx [30] adds a proximal term for the same reason. We use the Flower framework [6].

Federated and lightweight intrusion detection. DÏoT [42] trains per-device-type anomaly detectors across security gateways with FL; Belarbi et al. [5] study federated IDS on TON_IoT with one client per IP address; and FLAME [43] defends FL against backdoors, including on IoT IDS data. Closest to us, Li et al. [29] combine FL and TinyML for intrusion detection on N-BaIoT in an IoT/edge/cloud architecture, using an outlier-resistant scaler, feature reduction and quantization. We add an identically configured centralized and a local-only baseline on two datasets, a step-by-step analysis of compression without pooled data, missed-attack accounting, an analysis of quantization calibration, and the toolchain failures of Section 5. Outside FL, lightweight detectors run on gateways (Kitsune [37]) or programmable switches (HorusEye [12], NetBeacon [51]); we target the endpoint microcontroller.

TinyML and compression. TensorFlow Lite Micro (TFLM) [10] runs integer and float models on microcontrollers [3]; MCUNet [31] co-designs models and the inference engine, Lin et al. [33] train on 256 KB of memory, and SeeDot [17] and MinUn [25] compile fixed-point and mixed-precision models for microcontrollers. In FL, HeteroFL [11] trains width-reduced submodels and aggregates BatchNorm statistics after training, and FedTiny [23] prunes without pooled data during training; we compress an already-trained federated model and use its BatchNorm statistics for INT8 calibration. Pruning [20], distillation [21] and quantization [24] reduce model size; surveys and guidance on PTQ and QAT are given by Gholami et al. [15] and Nagel et al. [39], who also describe oscillation-driven QAT instability [40]. Section 4.5 reports a different, data-driven QAT failure.

Calibration and quantization ranges. Post-training calibration can clip activation ranges instead of using their extremes: percentile and entropy (KL) calibration [49], or the MSE-optimal clip of ACIQ [4]. Wu et al. also recommend calibrating first and then fine-tuning with QAT [49]. PACT [9] and LSQ [14] learn the clip or step size during QAT, and DFQ [41] quantizes without data by equalizing weight ranges across layers (CLE). Distillation and quantization have been combined before [44], and tfmot provides sparsity-preserving QAT because standard QAT re-densifies pruned weights [48]. In FL, quantization has mainly been used to compress updates [45]; Gupta et al. train federated models that stay accurate under later quantization [19]. Our input clipping is an instance of clipped calibration; Section 4.4 compares it with these estimators and with QAT-range, data-free and learned-clipping variants on our models.

Robustness of compressed models. FGSM [16] and PGD [35] are the standard gradient attacks. Quantization can amplify adversarial noise [32] or improve or degrade robustness depending on attack strength [18]; weight quantization [47] and deep quantization [2] interact with adversarial training. We measure whether the federated model's robustness carries over to its INT8 derivatives.

3 Method

3.1 Datasets and Preprocessing

CIC-IDS2017 [46]. The eight MachineLearningCSV files (2,830,743 flows, 78 numeric features). Labels are binarized (benign vs. any attack), $\pm\infty$ and non-numeric values are replaced, and duplicate rows are removed before any split (2,499,543 unique flows, 425,863 attacks). A stratified 80/20 split gives a test set of 499,909 flows (85,173 attacks) at the natural class ratio. On the training split only, the majority class is undersampled to at most 4:1, a StandardScaler is fitted, and SMOTE [7] balances the classes.

TON_IoT [38]. The network train/test file (211,043 records). We drop host identifiers and ephemeral ports (src_ip, dst_ip, src_port) to avoid shortcut features and keep dst_port. Text columns with at most 50 distinct values are label-encoded. The high-cardinality DNS query and URI fields are not used directly; instead we add six row-local features (length, digit count and non-alphanumeric count of each field), which cannot leak information across records, for 43 features. Deduplication on the other 37 features leaves 105,176 records (81,070 attacks); the test set has 21,036 records (16,215 attacks). The same balancing, scaling and SMOTE steps follow. The text-derived features and the TON_IoT focal-loss setting were selected on validation data (Appendix B).

3.2 Federated Training

The model is an MLP (512–256–128, ReLU, each hidden layer followed by BatchNorm and Dropout, sigmoid output). We train it with Flower [6] on four clients with FedAvgM (server momentum 0.5, server learning rate 0.1) [22], 60 rounds × 3 local epochs, batch size 128, Adam, and a server-broadcast cosine learning-rate schedule ($10^{-3} \rightarrow 10^{-4}$). Class imbalance is handled with focal loss [34] ($\alpha = 0.35$ for CIC-IDS2017; $\alpha = 0.5$ for TON_IoT, where attacks are the majority) and class weights. Clients receive either a near-IID partition (each class split evenly) or a Dirichlet($\alpha = 0.3$) label partition [22]. The centralized baseline trains the identical model and loss for the same 180 epochs.

3.3 FL-Faithful Compression

After training, the server holds only the global float model. (1) BatchNorm is folded exactly. Because it follows the ReLU, it is folded into the *next* Dense layer (Appendix A; maximum deviation $\leq 2 \cdot 10^{-5}$). (2) Structured pruning removes 50% of the neurons of each hidden layer by weight magnitude. (3) One participating client fine-tunes the pruned model for 3 epochs on 10,000 of its own samples. (4) The same client performs 2 epochs of QAT fine-tuning [24], and the model is exported as INT8 TFLite. We call this the *base recipe*. Fine-tuning on 10,000 pooled samples is reported only as an upper bound, because a federated server would not hold them.

3.4 Recall-Priority Deployment

For deployments that must miss as few attacks as possible we extend the base recipe in three ways that use only client-held data: (a) fine-tuning targets $0.5\,y + 0.5\,\sigma(z_{\text{teacher}}/2)$, i.e. distillation [21] from the federated model at temperature 2; (b) inputs clipped to ±5 standard deviations during fine-tuning and calibration; (c) optionally, federated fine-tuning, in which all clients fine-tune and the server averages the weights after every epoch. The decision threshold is the largest value that reaches a target attack recall on a held-out part of the fine-tuning client's data (20,000 samples for CIC-IDS2017, 8,000 for TON_IoT), disjoint from every fine-tuning draw. The test split is never used for selection.

3.5 Evaluation

Unless stated otherwise, metrics are computed on the test split at a fixed threshold of 0.3, chosen before these experiments. We report accuracy, precision, attack recall, F1, the false-alarm rate (FAR, the share of benign records flagged), and the number of missed attacks out of all test attacks. Robustness uses FGSM and PGD ($\epsilon = 0.1$ on standardized features; PGD with 10 steps); adversarial examples are computed on the float federated model and transferred to all other models. Where fine-tuning or calibration involves random draws, we repeat it three to five times and report the mean and range.

Setup. All runs used one desktop (16-core CPU under WSL2; Flower simulates the clients on CPU). A CIC-IDS2017 federated run takes about 5.4 h, a TON_IoT run about 25 min. Code, configurations, logs and per-run result files are in the artifact repository; Appendix D maps every table to its run.

4 Results

4.1 Federated Training and Where Federation Helps

Federated training comes within 1.65 F1 points of centralized training on CIC-IDS2017 and within 0.27 on TON_IoT (Table 1); on both datasets the gap lies mostly in precision, and all models detect more than 99.8% of attacks. A second, independent TON_IoT training run differs by 0.03 F1. A fixed learning rate does as well as the cosine schedule (F1 87.20 vs. 85.70 and 99.34 vs. 99.41), so we do not claim a benefit for the schedule. The Dirichlet partition is extreme: on CIC-IDS2017 one client holds 73% of the training data and two clients see almost no attacks. Federated training still loses only 0.11 F1 on CIC-IDS2017 and 0.81 on TON_IoT, where the model becomes recall-heavy (one missed attack, FAR 9.5%).

Table 2 shows where federation pays off. Clients whose local data contains few attacks detect only 45–93% of attacks when trained alone; on the same partitions the federated model detects 99.83% (CIC-IDS2017) and 99.99% (TON_IoT). Clients that already hold large, representative data gain little: on near-IID CIC-IDS2017 each client alone reaches F1 88.3–90.0 (federated: 85.70) at a precision-heavier operating point that misses 3.6–19 times as many attacks. The near-IID partition is therefore the setting in which federation is least needed. Local and federated training budgets are not matched, and at a fixed threshold the F1 differences among data-rich clients mainly reflect operating points.

4.2 Compression Without Pooled Data

INT8 PTQ alone gives 3.5× at a cost of about one F1 point for the calibration draw used in Table 3, but that cost varies widely between draws (§4.3). Pruning half of each hidden layer gives a further 3.5× only if the pruned model is fine-tuned; without fine-tuning it breaks down (F1 42.28 on CIC-IDS2017, FAR 30% on TON_IoT). With fine-tuning and QAT fine-tuning on one client's data, the INT8 model is 65.4 KB on CIC-IDS2017 (12.27×) and 56.7 KB on TON_IoT (12.92×) and stays close to the float model (F1 85.59 vs. 85.70 and 98.67 vs. 99.41). The QAT fine-tuning step matters: client-fine-tuned PTQ reaches only F1 90.20 on TON_IoT. Pooled fine-tuning scores a higher CIC-IDS2017 F1 only by moving toward precision; it misses 4,216 attacks instead of 291.

Which client fine-tunes. The fine-tuning client's class mix shifts the operating point. On the CIC-IDS2017 Dirichlet model, fine-tuning on client 3 (24.9% attacks) gives the highest F1 we observed (91.08, FAR 2.75%) but misses 4,415 attacks, fifteen times as many as the near-IID model fine-tuned on a 50%-attack client. On TON_IoT, client 2 of the Dirichlet run gives F1 98.80 (248 missed). *Source: runs E, V.*

Compression strength. Figure 1 sweeps the pruning ratio from 30% to 90%. Without fine-tuning, pruning collapses from 50% (CIC-IDS2017) or 85% (TON_IoT) onward. With client fine-tuning, CIC-IDS2017 degrades gracefully (70%: 31 KB, F1 84.4–85.0; 90%: 9.9 KB, F1 78.0), and on TON_IoT client fine-tuning + QAT holds F1 98.65–98.74 down to 8.2 KB (90× smaller than FP32), whereas client fine-tuning + PTQ stays at 87.7–91.5. QAT fine-tuning is the choice that is reliable on both datasets.

Table 1. Federated vs. centralized training and the effect of a Dirichlet(0.3) partition (float models, test split, threshold 0.3). *Source: CIC-IDS2017 runs B, I; TON_IoT runs S, U.*

Dataset	Model	Accuracy	Precision	Attack recall	F1	FAR	Missed / all attacks
CIC-IDS2017	Centralized	95.07%	77.60%	99.90%	87.35	5.92%	88 / 85,173 (0.10%)
	Federated, near-IID	94.32%	75.01%	99.94%	85.70	6.84%	52 / 85,173 (0.061%)
	Federated, fixed learning rate	95.01%	77.40%	99.84%	87.20	5.99%	136 / 85,173 (0.16%)
	Federated, Dirichlet(0.3)	94.27%	74.91%	99.83%	85.59	6.87%	144 / 85,173 (0.17%)
TON_IoT	Centralized	99.51%	99.43%	99.94%	99.68	1.93%	10 / 16,215 (0.062%)
	Federated, near-IID	99.08%	98.94%	99.88%	99.41	3.61%	19 / 16,215 (0.12%)
	Federated, second training run	99.04%	98.91%	99.86%	99.38	3.71%	23 / 16,215 (0.14%)
	Federated, fixed learning rate	98.97%	98.82%	99.86%	99.34	4.00%	23 / 16,215 (0.14%)
	Federated, Dirichlet(0.3)	97.81%	97.25%	99.99%	98.60	9.52%	1 / 16,215 (0.006%)

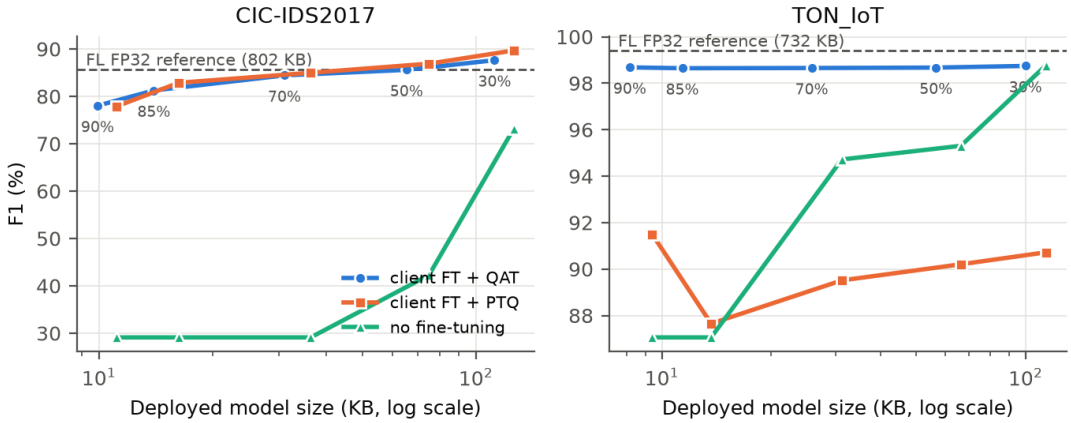


Figure 1. F1 vs. deployed INT8 size for structured pruning ratios 30–90% (labels), with client fine-tuning followed by QAT fine-tuning or PTQ, and without fine-tuning. Dashed: uncompressed federated model. *Source: CIC-IDS2017 run G; TON_IoT run W.*

Table 2. Each client trained alone (10 epochs, ≤200k samples) vs. the federated model of the same partition; recall and missed attacks on the test split. *Source: runs K, V; federated rows as Table 1.*

Client (attack share)	Recall alone	Missed alone	Federated
CIC-IDS2017 near-IID, each (50%)	98.85–99.78%	185–982	52
CIC-IDS2017 Dir. client 0 (0.1%)	44.97%	46,872	144
CIC-IDS2017 Dir. client 2 (2.6%)	91.11%	7,572	
TON_IoT near-IID, each (50%)	99.69–99.81%	31–51	19
TON_IoT Dir. client 0 (0.4%)	85.53%	2,346	1
TON_IoT Dir. client 3 (8.5%)	93.39%	1,072	

4.3 Quantization Methods and Calibration

Every method that keeps activations in float or 16 bits loses at most 0.25 F1 (Table 4). Full-integer INT8 is the only lossy method (TON_IoT pruned model: 9.2 F1, FAR 2.0%→13.2%); since int16x8 uses the same INT8 weights and recovers, the loss comes from 8-bit activations. The TFLM build we deploy runs only FP32 and full-integer INT8 kernels, so dynamic range (the best size–accuracy point on CIC-IDS2017) suits Linux-class gateways but not the microcontroller.

Full-integer PTQ is also unstable (Table 5). The calibration source (pooled data or any client) does not matter, but the random draw does: the same TON_IoT model ranges from F1 32.7 to 99.4 and the CIC-IDS2017 model from 29.2 to 85.0, and more calibration data makes it worse. The TFLite converter sets each activation range from the minimum and maximum it sees, and a larger calibration set is more likely to contain an extreme record of the heavy-tailed standardized features: the worst conversions have hidden-layer maxima near 400 (TON_IoT) and 9,609 (CIC-IDS2017), against roughly 50–210 for most others. A one-line change removes the instability: clipping calibration inputs to ±5 standard deviations gives F1 99.19 for every TON_IoT set and 83.0–84.9 on CIC-IDS2017 (±10 works as well; ±3 is too tight). QAT fine-tuning is more stable, though not draw-independent (§4.4 explains why): over five fine-tuning draws the base recipe gives F1 98.79–99.51 on TON_IoT (PTQ: 96.89–99.19) and 86.77–89.28

Table 3. Compression of the near-IID federated model. "Client FT" uses 10,000 samples of client 0's own data; "pooled" uses 10,000 pooled samples a federated server would not hold. The rows without fine-tuning calibrate INT8 on 500 pooled samples (one draw; see §4.3). *Source: CIC-IDS2017 run E; TON_IoT run V.*

	CIC-IDS2017			TON_IoT		
Variant	Size	F1 / FAR	Missed of 85,173	Size	F1 / FAR	Missed of 16,215
FP32 federated model (TFLite)	802.5 KB	85.70 / 6.84%	52	732.4 KB	99.41 / 3.61%	19
INT8 PTQ only	226.9 KB	84.75 / 7.37%	74	209.4 KB	98.96 / 3.86%	153
Prune 50%, no fine-tuning → PTQ	75.1 KB	42.28 / 56.04%	46	66.3 KB	95.30 / 30.20%	129
Prune 50% → client FT → PTQ	75.1 KB	86.90 / 6.08%	346	66.3 KB	90.20 / 13.23%	2,369
Prune 50% → client FT → QAT FT (base recipe)	65.4 KB	85.59 / 6.82%	291	56.7 KB	98.67 / 2.26%	319
Same, pooled fine-tuning (upper bound)	65.4 KB	90.72 / 2.98%	4,216	56.7 KB	99.16 / 3.48%	105

Table 4. Post-training quantization methods: size and F1 (calibration: one draw of 500 client-0 samples). "Pruned" = prune 50% → client FT. TFLM = runs on the TFLM build we deploy. *Source: CIC-IDS2017 run J; TON_IoT run V.*

		CIC-IDS2017				TON_IoT			
		Federated		Pruned + client FT		Federated		Pruned + client FT	
Method	TFLM	Size	F1	Size	F1	Size	F1	Size	F1
FP32	yes	802 KB	85.70	243 KB	89.39	732 KB	99.41	208 KB	99.42
Dynamic range (INT8 weights, float compute)	no	216 KB	85.58	69 KB	90.13	199 KB	99.41	61 KB	99.43
Float16 weights	no	404 KB	85.70	124 KB	89.35	369 KB	99.41	106 KB	99.42
Full-integer INT8	yes	227 KB	85.02	75 KB	86.90	209 KB	97.69	66 KB	90.20
int16x8 (INT8 weights, 16-bit activations)	untested	238 KB	85.48	81 KB	89.61	220 KB	99.40	72 KB	99.41

Table 5. Full-integer PTQ of the federated models over 15 calibration sets per size (5 sources × 3 random draws): median F1, minimum F1, and sets within 1 F1 of FP32. *Source: CIC-IDS2017 runs L, M; TON_IoT run V.*

	TON_IoT (FP32 99.41)			CIC-IDS2017 (FP32 85.70)		
Calibration	Med.	Min.	≤1	Med.	Min.	≤1
100 samples	99.36	96.11	14/15	84.60	81.47	5/15
500 samples	99.37	54.01	13/15	83.11	80.91	2/15
2,000 samples	97.68	32.72	5/15	82.60	29.16	1/15
2,000, clipped ±5	99.19	99.19	15/15	84.61	83.02	4/15

on CIC-IDS2017 (PTQ: 79.37–89.05). Table 3's CIC-IDS2017 value (85.59) lies below all five draws, so it is conservative rather than selected.

4.4 Calibration Estimators, QAT Ranges and Data-Free Calibration

To compare range estimators with everything else fixed, we set the activation ranges of the tfmot fake-quantization model directly and convert it; with min/max ranges this reproduces TFLite's per-tensor PTQ exactly. TFLite's default PTQ uses per-channel weight scales, QAT exports per-tensor.

Estimators. Only estimators that clip hard remove the collapses: the 99.9th percentile and our ±5 SD input clip. The 99.99th percentile, the MSE-optimal clip and KL calibration still fail on some calibration sets (minimum F1 29–44 on CIC-IDS2017, 35–97 on TON_IoT). The rare extreme records that stretch the ranges are too rare to move a per-tensor MSE or KL objective. With the same per-tensor weights, the 99.9th percentile is at least as good as our clip (TON_IoT FAR 3.55% vs. 4.94%, CIC-IDS2017 misses 58 vs. 239). Per-channel weights barely change min/max calibration, but with clipped inputs they cut CIC-IDS2017 misses from 239 to 68; CLE recovers the same effect for per-tensor weights (58).

Data-free calibration. The server can calibrate without any client data. Because inputs are standardized, synthetic $\mathcal{N}(0, I)$ inputs are a usable calibration set, and the federated BatchNorm statistics (averaged by FedAvg like any weight) give each post-ReLU activation's mean and standard deviation; we set the range to $\max_c(\mu_c + k\sigma_c)$. Both match clipped real-data calibration on both datasets, and CLE plus BN statistics ($k = 6$) gives the best CIC-IDS2017 PTQ result (84.64, 55 misses). $k = 3$ is too tight (CIC-IDS2017 532, TON_IoT 282 misses).

QAT ranges and order. tfmot's default QAT does not learn its ranges in a fine-tuning run this short. Activation ranges start at ±6 and follow an exponential moving average (decay 0.999), so after our ≈140 steps about 87% of the start remains: hidden ranges end near $[-5.2, 10]$ although ReLU outputs are non-negative. The input quantizer instead

Table 6. Full-integer PTQ of the unpruned federated models with different range estimators: F1 mean [minimum], missed attacks (mean), FAR (mean). Real-data rows: the 15 calibration sets of Table 5 (2,000 samples). Data-free rows use no client data (synthetic: 3 draws of 2,000 $\mathcal{N}(0, I)$ inputs; BN: ranges $\mu + k\sigma$ from the federated BatchNorm statistics). CLE = cross-layer equalization [41]. *Source: runs X1, X3.*

		CIC-IDS2017 (85,173 attacks)			TON_IoT (16,215 attacks)		
Weights	Range estimator	F1 [min]	Missed	FAR	F1 [min]	Missed	FAR
per-channel	min/max (TFLite default)	77.67 [29.16]	349	15.51	77.91 [32.72]	4,779	2.99
per-channel	inputs clipped ±5 SD (ours)	84.38 [83.02]	68	7.58	99.19 [99.19]	27	4.94
per-tensor	min/max	76.64 [29.20]	397	16.14	81.00 [36.06]	4,156	4.48
per-tensor	percentile 99.9 [49]	83.16 [82.11]	58	8.30	**99.39 [99.38]**	26	3.55
per-tensor	percentile 99.99	79.27 [31.51]	543	13.34	99.31 [98.81]	50	3.62
per-tensor	MSE-optimal clip (ACIQ objective [4])	77.28 [29.28]	630	15.50	88.68 [34.62]	2,395	4.72
per-tensor	KL / entropy [49]	80.93 [44.16]	565	10.68	98.97 [96.69]	131	4.20
per-tensor	inputs clipped ±5 SD	82.93 [82.09]	239	8.37	99.19 [99.18]	27	4.94
per-tensor	CLE + inputs clipped ±5 SD (3 sets)	84.46 [84.04]	58	7.54	99.18 [99.18]	29	4.93
per-channel	data-free: synthetic $\mathcal{N}(0, I)$ inputs	82.22 [80.58]	54	8.87	99.16 [99.08]	29	5.06
per-tensor	data-free: BN statistics, $k = 4$	83.03	72	8.37	99.17	26	5.06
per-tensor	data-free: BN statistics, $k = 6$	82.51	66	8.69	99.26	27	4.44
per-tensor	data-free: CLE + BN statistics, $k = 6$	**84.64**	55	7.44	99.25	30	4.48
	FP32 federated model	85.70	52	6.84	99.41	19	3.61

Table 7. QAT ranges on the base recipe (prune 50%, client FT, 2 QAT epochs, 5 FT draws): F1 mean [range], missed attacks (mean). "Start": ranges initialized from client calibration and then updated by tfmot (5 estimators, range of means); "fixed": held at the calibrated values. *Source: runs X2, X4.*

	CIC-IDS2017		TON_IoT	
Variant	F1	Missed	F1	Missed
PTQ, min/max (TFLite)	85.81 [79.4–89.1]	562	98.62 [96.9–99.2]	351
QAT, tfmot default	88.02 [86.8–89.3]	513	99.00 [98.8–99.5]	204
QAT, calibrated start	87.79–88.18	447–664	98.99–99.03	194–208
QAT, fixed min/max	**89.49** [87.4–91.4]	757	99.46 [99.4–99.5]	74
QAT, fixed pct. 99.99	88.58 [87.4–90.0]	**392**	**99.47** [99.4–99.5]	**70**
QAT, fixed MSE	88.62 [87.3–90.1]	839	99.47 [99.4–99.6]	76
QAT, fixed ±5 SD	88.10 [87.4–89.0]	598	99.34 [99.2–99.4]	79
QAT, learned clip (PACT-style)	87.67–88.73	652–718	98.99–99.00	199–208

tracks the minimum and maximum of every fine-tuning input (CIC-IDS2017: up to 61 SD, TON_IoT: 85 SD, against 12 and 27 at the 99.99th percentile). Starting QAT from calibrated ranges, the PTQ-then-QAT order of Wu et al. [49], changes the hidden ranges but keeps this input range, and gives the default's results. Holding all ranges at the calibrated values also keeps the input range, and it is the only variant that helps: TON_IoT misses drop from 204 to 70 (FAR 2.49%⟶2.14%), CIC-IDS2017 misses from 513 to 392. Learning the hidden clips (PACT-style [9], trained in log space) does not help. The bottleneck is therefore the input range, not the hidden activations. This also explains why QAT on clipped inputs was the best TON_IoT pipeline in Table 9 and why training-time QAT collapses on CIC-IDS2017 (§4.5).

4.5 Training-Time QAT Inside FL: a Negative Result

With fake quantization from the first federated round, the CIC-IDS2017 model collapses to 34.74% accuracy (F1 34.30, FAR 78.7%); its float weights reach only F1 46.8. The learned input quantizer range is $[-1231, 405]$ and activation ranges reach 2,342, so one INT8 step spans about 6.4 standard deviations and ordinary inputs collapse to the zero point. On TON_IoT, whose features are less extreme, the same procedure works (accuracy 98.14%, F1 98.79). We therefore quantize after training. *Source: CIC-IDS2017 run A; TON_IoT run U.*

4.6 Client-Local Distillation

A client can also train a narrow student on its own data, distilled from the federated model (Table 8). On TON_IoT the 1/8-width distilled student keeps F1 99.21 at 11.6 KB with plain PTQ (63× smaller than FP32), and distillation makes students more robust to INT8 PTQ (at most 0.29 F1 lost vs. 0.55–0.70 from scratch). On CIC-IDS2017 distilled students inherit the teacher's recall-heavy operating point (recall 99.5–99.7%, FAR 5.6–6.5%), so scratch students score a higher F1 at the fixed threshold; the 91.10 outlier misses 3,137 attacks and is not a better detector.

4.7 Combining Pruning, PTQ, QAT and Distillation

Table 9 compares complete pipelines (three further variants are in the artifact). Only structured pruning shrinks what the microcontroller stores: TFLM keeps weights dense, so magnitude pruning leaves flash at 189–227 KB even at 80% sparsity

Table 8. Students trained on client 0's data, F1 as FP32 / INT8 PTQ / INT8 after QAT fine-tuning. KD: distilled from the federated model; scratch: hard labels only. *Source: CIC-IDS2017 run J; TON_IoT run V.*

Student width	INT8 size (CIC-IDS2017/ TON_IoT)	CIC-IDS2017 KD	CIC-IDS2017 scratch	TON_IoT KD	TON_IoT scratch
1/2 (256–128–64)	75 / 66 KB	87.90 / 87.64 / 87.36	89.75 / 89.36 / 87.21	99.46 / 99.37 / 99.05	99.42 / 98.81 / 99.08
1/4 (128–64–32)	29 / 25 KB	87.31 / 87.37 / 86.42	88.64 / 88.74 / 86.46	99.42 / 99.13 / 98.96	99.43 / 98.73 / 99.05
1/8 (64–32–16)	14 / 12 KB	86.30 / 86.07 / 86.55	88.29 / 88.51 / 91.10	99.20 / 99.21 / 98.78	99.37 / 98.82 / 99.00

Table 9. Complete pipelines from the near-IID federated model using only client 0's data; mean and range over three fine-tuning draws. Flash = TFLite file stored by TFLM; gzip = compressed transfer size. Clipping at ±5 standard deviations. *Source: CIC-IDS2017 run N; TON_IoT run V.*

Pipeline	Flash KB (TON_IoT/CIC-IDS2017)	gzip KB	TON_IoT F1	CIC-IDS2017 F1	CIC-IDS2017 missed of 85,173
FP32 federated model	732 / 803	684 / 755	99.41	85.70	52
INT8 PTQ	209 / 227	160 / 150	99.10 (98.92–99.40)	82.36 (81.04–83.21)	163
INT8 PTQ, clipped calibration	209 / 227	160 / 150	99.18 (99.18–99.19)	83.97 (83.27–84.35)	57
QAT fine-tuning	189 / 207	131 / 99	99.22 (99.07–99.52)	90.51 (89.27–91.99)	580
Structured 50% → FT → PTQ	66 / 75	49 / 47	98.32 (96.89–99.19)	84.39 (79.37–87.27)	500
Structured 50% → FT → PTQ, clipped	66 / 75	49 / 47	99.42 (99.40–99.44)	87.67 (86.86–88.09)	830
Structured 50% → FT → QAT (base recipe)	57 / 65	40 / 34	99.12 (98.89–99.51)	88.37 (87.64–89.28)	498
Structured 50% → FT → QAT on clipped inputs	57 / 65	41 / 35	99.49 (99.47–99.51)	90.36 (88.54–92.44)	1,163
Structured 50% → KD FT → QAT	57 / 65	40 / 34	99.19 (99.01–99.49)	86.04 (85.61–86.56)	274
QAT first → structured 50% → FT → QAT	57 / 65	41 / 34	99.17 (98.97–99.48)	88.64 (86.65–90.58)	687
Magnitude 50% → PTQ, clipped	209 / 227	127 / 128	99.47 (99.46–99.47)	89.72 (89.48–89.95)	491
Magnitude 80% → PQAT	189 / 207	58 / 57	99.03 (98.82–99.43)	89.77 (88.99–91.10)	609
Magnitude 80% → standard QAT	189 / 207	80 / 62	99.06 (98.86–99.46)	89.77 (89.31–90.52)	571
Structured 50% → magnitude 50% → PQAT	56 / 65	33 / 31	99.01 (98.74–99.46)	88.66 (88.05–89.86)	699

and only reduces the gzip size, which matters for transferring updates. Sparsity-preserving QAT (PQAT) keeps the target sparsity, whereas standard QAT lets pruned weights grow back (80%→60% zeros on TON_IoT). Clipping helps every TON_IoT pipeline: QAT fine-tuning on clipped inputs slightly exceeds the FP32 F1 at 57 KB (99.49) and misses 84 attacks on average vs. 174 for the base recipe. On CIC-IDS2017, clipped calibration removes the PTQ collapses, but after pruning clipping raises F1 by trading recall for precision (1,163 missed vs. 498), while distillation fine-tuning does the opposite (274 missed). Quantizing before pruning changes nothing beyond the draw-to-draw spread. On CIC-IDS2017 no pipeline dominates at the fixed threshold; the choice is an operating point, which §4.8 makes explicit with a client-chosen threshold.

4.8 Recall-Priority Deployment

Lowering the threshold of the base-recipe model alone reaches the FP32 model's missed-attack count, but only at a 23% (CIC-IDS2017) or 38% (TON_IoT) FAR (Table 10). The hard-label QAT model pushes most scores into the lowest INT8 output levels: the selected thresholds are one to ten steps of the output's 1/256 resolution, so the threshold cannot be tuned finely. Distilled students keep their scores in the middle of the range (thresholds 0.15–0.26), where the INT8 output has resolution, and clipping separates the classes further. Together they let the 65 KB CIC-IDS2017 model miss 24–27 of 85,173 attacks at an 11–14% FAR; federated fine-tuning gives the better trade-off (27 at 11.3%). Pruning less trades flash for false alarms: at 112 KB the model misses 23 attacks at 9.7%, and at 207 KB it reaches the FP32 model's miss count at a lower FAR (51 at 6.7% vs. 52 at 6.8%). On TON_IoT, federated fine-tuning also makes the 57 KB model stable across draws: 12 of 16,215 attacks missed (at most 14) at 6.6% FAR (at most 6.8%), or 30 at 3.7%. A 4× attack weight, pure teacher targets, temperature 4 and five times more fine-tuning data did not improve these trade-offs. Neither did the levers of §4.4: fixed QAT ranges only move the operating point (CIC-IDS2017 112 KB: 28.7 missed at 8.6% FAR vs. 23.3 at 9.7%; TON_IoT: 8.3 at 7.1% vs. 12.0 at 6.6%), and CLE hurts CIC-IDS2017, because the recipe already clips the fine-tuning inputs and so bounds the input range. *Source: runs X5, X6.*

The client's held-out data is a reliable guide for FAR (within 0.35 points of the test FAR per draw on CIC-IDS2017 and 0.7 on TON_IoT). Recall targets up to 99.95% are met on CIC-IDS2017; very high targets need more validation attacks than one client holds (TON_IoT: 99.82–99.93% on test for 99.9–99.99% targets). *Source: runs O, P, T.*

Table 11 shows where the ceiling is. The FP32 federated model itself misses 7 of the 11 Infiltration test flows at every threshold we evaluated; Infiltration has only 25 training flows. Compression adds one loss of its own: the 65 KB models miss on average 3.3 of the 5 Heartbleed flows (6 training flows), which the 112 KB model keeps. Fewer misses would

Table 10. Recall-priority deployment (§3.4); means over three fine-tuning draws. "Client val.": threshold chosen on held-out client-0 data for the stated recall. *Source: CIC-IDS2017 runs O, P; TON_IoT run T.*

Model	Threshold selection	Threshold	Missed / all	Missed share	Worst draw	FAR
CIC-IDS2017 (85,173 test attacks)						
FP32 federated, 803 KB	fixed	0.30	52	0.061%	–	6.84%
FP32 federated, 803 KB	client val., ≥99.99%	0.269	37	0.043%	–	7.56%
65 KB, base recipe	fixed	0.30	463	0.54%	483	5.97%
65 KB, base recipe	client val., ≥99.95%	0.005	44	0.052%	58	23.39%
65 KB, distillation + clipped inputs	client val., ≥99.95%	0.198	44	0.052%	55	10.43%
65 KB, distillation + clipped inputs	client val., ≥99.99%	0.171	24	0.028%	25	14.33%
65 KB, + federated fine-tuning	client val., ≥99.99%	0.180	27	0.032%	33	11.28%
112 KB (prune 30%), distillation + clipped	client val., ≥99.99%	0.189	23	0.027%	27	9.65%
207 KB (no pruning), distillation + clipped	client val., ≥99.95%	0.229	51	0.060%	52	6.71%
TON_IoT (16,215 test attacks)						
FP32 federated, 732 KB	fixed	0.30	19	0.12%	–	3.61%
FP32 federated, 732 KB	client val., ≥99.95%	0.259	9	0.056%	–	4.71%
57 KB, base recipe	client val., ≥99.9%	0.038	15	0.094%	16	37.65%
57 KB, distillation + clipped inputs	client val., ≥99.99%	0.165	4	0.023%	8	12.30%
57 KB, + federated fine-tuning	client val., ≥99.9%	0.244	30	0.19%	32	3.68%
57 KB, + federated fine-tuning	client val., ≥99.99%	0.203	12	0.074%	14	6.61%
100 KB (prune 30%), distillation + clipped	client val., ≥99.99%	0.186	5	0.031%	8	9.07%

Table 11. Missed CIC-IDS2017 test flows per attack type at the 99.99% client-validation target (compressed models: mean over three draws). Types missed by none of the models are omitted. *Source: runs P, Q.*

Attack type	Test flows	FP32	65 KB fed. FT	112 KB
Infiltration	11	7	8.3	6.3
Heartbleed	5	0	3.3	0
SQL injection	4	1	0	0.3
Bot	368	11	6.0	6.7
Web brute force	301	2	1.3	1.0
PortScan	18,282	3	3.7	3.3
DDoS	25,569	8	1.3	1.7
Other DoS / Patator	6,026	5	3.0	4.0
All attacks	85,173	37	27	23

therefore require a better federated model (for example class-aware objectives for rare attacks or corrected labels [13]), not a better compression pipeline. We deploy the 112 KB CIC-IDS2017 model (first draw: 27 missed, FAR 10.0%, threshold 0.191) and the 57 KB TON_IoT model with federated fine-tuning (first draw: 12 missed, FAR 6.5%, threshold 0.207); compared with the base recipe at threshold 0.3 (291 and 319 missed), they miss 11–27 times fewer attacks at 1.5–2.9 times the FAR.

4.9 Adversarial Robustness

On CIC-IDS2017, INT8 PTQ preserves the federated model's robustness; pruning with fine-tuning reduces it, and the QAT-fine-tuned model is clearly more robust than the PTQ one, echoing the quantization–robustness interaction reported in [2, 18, 47] (Table 12). The centralized model is markedly more fragile although it is attacked only by transfer. On TON_IoT, $\epsilon = 0.1$ drives every model except the pooled upper bound to predict all perturbed records as benign. These are unconstrained L_∞ perturbations that ignore feature semantics, so they overstate what a network attacker can realize.

4.10 On-Device Measurement

We ran both deployment models and their FP32 federated models on an ESP32 (ESP32-D0WD-V3, Xtensa LX6 at 240 MHz, ESP-IDF v4.4.7; protocol in Appendix C). On fixed inputs every model reproduces the host interpreter (FP32 exactly, INT8 within one output step), and all decisions agree at each model's threshold. Over 100 inferences the CIC-IDS2017 112 KB INT8 model takes 26.3 ms (FP32: 114.6 ms, 4.4×) and the TON_IoT 57 KB INT8 model 13.3 ms (FP32: 104.7 ms, 7.9×); the 95th percentile is within 0.05 ms of the mean, and tensor arenas need 1.4–3.9 KB. One core thus classifies about 38 (CIC-IDS2017) or 75 (TON_IoT) flows per second. *Source: ESP32 run, data/processed/ablation/esp32_benchmark.json.*

5 Four Silent Failure Modes

An earlier version of this study reported compressed-model results that turned out to be wrong. Re-running the experiments exposed four errors in our pipeline. None raised an exception, and each produced plausible numbers (Table 13). The same components (Flower, TensorFlow Model Optimization, Keras BatchNorm) are widely used, and empirical studies

Table 12. Accuracy (%) on 20,000 test records before and after FGSM and PGD ($\epsilon = 0.1$), perturbations computed on the federated FP32 model and transferred to the other models. On TON_IoT, 23.0% equals the benign share of the test set: those models label every perturbed record benign. *Source: CIC-IDS2017 run F; TON_IoT run W.*

Model	CIC-IDS2017 clean	CIC-IDS2017 FGSM	CIC-IDS2017 PGD	TON_IoT clean	TON_IoT FGSM	TON_IoT PGD
Federated FP32 (attack source)	94.3	68.6	55.6	99.1	23.0	22.9
Centralized FP32 (transfer)	95.1	30.8	29.0	99.5	23.0	23.0
INT8 PTQ only	93.9	70.4	60.4	98.4	23.0	22.9
Prune 50% → client FT → PTQ	94.8	32.5	23.9	85.7	23.0	23.0
Base recipe (client QAT FT)	94.2	44.3	45.6	98.0	23.0	23.0
Pooled QAT FT (upper bound)	96.3	44.7	46.2	98.7	91.1	91.1

Table 13. Silent failures found in our TensorFlow / tfmot / Flower pipeline and the checks we now run.

Failure	Symptom and check
Weight exchange without scales	Clients sent int8 codes without per-tensor scales; the server averaged codes as weights, rescaling every tensor to $\max\|w\| = 127$. Check: $\mathrm{deq}(\mathrm{q}(w)) \approx w$ at the receiver.
QAT stripping that copies nothing	Positional weight copy from the QAT wrapper failed silently; the "stripped" model kept its random initialization. Check: copy by name, refuse if nothing is copied, compare outputs.
BatchNorm folded on the wrong side	Folding post-ReLU BatchNorm into the preceding Dense flipped 11.6% of decisions. Check: fold into the next layer and assert equivalence.
Pruning resets BatchNorm statistics	Re-created BN layers lost their statistics (accuracy 95.4%→32.7%). Check: fold BN before pruning.

report that deployment and model-conversion faults often produce wrong models without errors [8, 26].

What exposed them was an invariant: the uncompressed federated model, evaluated at every stage of the compression pipeline (the FP32 rows of Table 3), must reproduce the float model's metrics. In the erroneous pipeline it was at chance level. We recommend this check for any FL-plus-compression study.

6 Limitations

Seeds: federated training ran once per configuration except for a second TON_IoT run (0.03 F1 apart); compression results are repeated over three to five draws. *Scale*: all experiments use four clients and one MLP architecture. *Comparisons*: we compare only against our own centralized baseline, because published CIC-IDS2017 results mostly use multiclass metrics, different splits and no deduplication, and the dataset has known labelling issues [13]. *Thresholds*: client validation sets are SMOTE-augmented, and recall targets above 99.95% need larger, non-augmented validation sets. *TON_IoT recipe*: the variants of Appendix B were designed after the initial TON_IoT model showed room for improvement, although the choice among them used validation data only. *Threat model*: attacks are feature-space perturbations transferred from the float model, not computed through the integer model or realized as traffic; training-time poisoning and backdoors [43] are out of scope, and the supervised detector does not target unseen attack classes as unsupervised detectors do [12, 37]. *Partitions*: near-IID and Dirichlet splits are synthetic; per-host splits [5] are more realistic. *Hardware*: latency is measured on one ESP32 board with the kernels of the TensorFlowLite_ESP32 1.0.0 library; optimized kernels and other microcontrollers would change the absolute numbers.

7 Conclusion

A federated MLP intrusion detector comes close to centralized training on CIC-IDS2017 and TON_IoT and loses less than one F1 point under a strongly non-IID partition, and federation matters most for clients with few local attacks. It can be compressed to a 65 KB INT8 TFLM model without pooled data, and with distillation, clipped inputs, federated fine-tuning and a client-chosen threshold the compressed models miss fewer attacks than the FP32 model at its default threshold (27 vs. 52 of 85,173 on CIC-IDS2017; 12 of 16,215 on TON_IoT). The remaining misses lie in the rarest attack classes, which the FP32 model also misses. Training-time QAT inside FL fails on heavy-tailed features, full-integer PTQ is unstable across calibration draws unless its inputs are clipped (the 99.9th-percentile estimator and calibration from federated BatchNorm statistics, which needs no client data, work as well), and an invariant check on the uncompressed federated model catches the silent toolchain failures that had invalidated our earlier results.

References

[1] Luigi Atzori, Antonio Iera, and Giacomo Morabito. 2010. The Internet of Things: A Survey. *Computer Networks* 54, 15 (2010), 2787–2805. https://doi.org/10.1016/j.comnet.2010.05.010

[2] Ferheen Ayaz, Idris Zakariyya, José Cano, Sye Loong Keoh, Jeremy Singer, Danilo Pau, and Mounia Kharbouche-Harrari. 2023. Improving Robustness Against Adversarial Attacks with Deeply Quantized Neural Networks. In *International Joint Conference on Neural Networks (IJCNN)*. https://doi.org/10.1109/IJCNN54540.2023.10191429

[3] Colby R. Banbury, Vijay Janapa Reddi, Max Lam, William Fu, Amin Fazel, Jeremy Holleman, Xinyuan Huang, Robert Hurtado, David Kanter, Anton Lokhmotov, David Patterson, Danilo Pau, Jae sun Seo, Jeff Sieracki, Urmish Thakker, Marian Verhelst, and Poonam Yadav. 2020. Benchmarking TinyML Systems: Challenges and Direction. *arXiv preprint arXiv:2003.04821* (2020).

[4] Ron Banner, Yury Nahshan, and Daniel Soudry. 2019. Post Training 4-bit Quantization of Convolutional Networks for Rapid-Deployment. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[5] Othmane Belarbi, Theodoros Spyridopoulos, Eirini Anthi, Ioannis Mavromatis, Pietro Carnelli, and Aftab Khan. 2023. Federated Deep Learning for Intrusion Detection in IoT Networks. In *IEEE Global Communications Conference (GLOBECOM)*. 237–242. https://doi.org/10.1109/GLOBECOM54140.2023.10437860

[6] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchi Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, and Nicholas D. Lane. 2020. Flower: A Friendly Federated Learning Research Framework. *arXiv preprint arXiv:2007.14390* (2020).

[7] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research* 16 (2002), 321–357. https://doi.org/10.1613/jair.953

[8] Zhenpeng Chen, Huihan Yao, Yiling Lou, Yanbin Cao, Yuanqiang Liu, Haoyu Wang, and Xuanzhe Liu. 2021. An Empirical Study on Deployment Faults of Deep Learning Based Mobile Applications. In *IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*.

[9] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. 2018. PACT: Parameterized Clipping Activation for Quantized Neural Networks. *arXiv preprint arXiv:1805.06085* (2018).

[10] Robert David, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, Ian Nappier, Meghna Natraj, Tiezhen Wang, Pete Warden, and Rocky Rhodes. 2021. TensorFlow Lite Micro: Embedded Machine Learning for TinyML Systems. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 3. 800–811.

[11] Enmao Diao, Jie Ding, and Vahid Tarokh. 2021. HeteroFL: Computation and Communication Efficient Federated Learning for Heterogeneous Clients. In *International Conference on Learning Representations (ICLR)*.

[12] Yutao Dong, Qing Li, Kaidong Wu, et al. 2023. HorusEye: A Realtime IoT Malicious Traffic Detection Framework using Programmable Switches. In *32nd USENIX Security Symposium*.

[13] Gints Engelen, Vera Rimmer, and Wouter Joosen. 2021. Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study. In *IEEE Security and Privacy Workshops (SPW)*. 7–12. https://doi.org/10.1109/SPW53761.2021.00009

[14] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. 2020. Learned Step Size Quantization. In *International Conference on Learning Representations (ICLR)*.

[15] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2021. A Survey of Quantization Methods for Efficient Neural Network Inference. *arXiv preprint arXiv:2103.13630* (2021).

[16] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2015. Explaining and Harnessing Adversarial Examples. In *International Conference on Learning Representations (ICLR)*.

[17] Sridhar Gopinath, Nikhil Ghanathe, Vivek Seshadri, and Rahul Sharma. 2019. Compiling KB-Sized Machine Learning Models to Tiny IoT Devices. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.

[18] Micah Gorsline, James Smith, and Cory Merkel. 2021. On the Adversarial Robustness of Quantized Neural Networks. In *Proceedings of the Great Lakes Symposium on VLSI (GLSVLSI)*. arXiv:2105.00227.

[19] Kartik Gupta, Marios Fournarakis, Matthias Reisser, Christos Louizos, and Markus Nagel. 2023. Quantization Robust Federated Learning for Efficient Inference on Heterogeneous Devices. *Transactions on Machine Learning Research (TMLR)* (2023). arXiv:2206.10844.

[20] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In *International Conference on Learning Representations (ICLR)*.

[21] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531* (2015).

[22] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. 2019. Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification. *arXiv preprint arXiv:1909.06335* (2019).

[23] Hong Huang, Lan Zhang, Chaoyue Sun, Ruogu Fang, Xiaoyong Yuan, and Dapeng Wu. 2023. Distributed Pruning Towards Tiny Neural Networks in Federated Learning. In *IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*. 190–201.

[24] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2704–2713.

[25] Shikhar Jaiswal, Rahul Kranti Kiran Goli, Aayan Kumar, Vivek Seshadri, and Rahul Sharma. 2023. MinUn: Accurate ML Inference on Microcontrollers. In *Proceedings of the 24th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES)*. 26–39.

[26] Purvish Jajal, Wenxin Jiang, Arav Tewari, Erik Kocinare, Joseph Woo, Anusha Sarraf, Yung-Hsiang Lu, George K. Thiruvathukal, and James C. Davis. 2024. Interoperability in Deep Learning: A User Survey and Failure Analysis of ONNX Model Converters. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. https://doi.org/10.1145/3650212.3680374

[27] Jakub Konečný, H. Brendan McMahan, Felix X. Yu, Peter Richtárik, Ananda Theertha Suresh, and Dave Bacon. 2016. Federated Learning: Strategies for Improving Communication Efficiency. *arXiv preprint arXiv:1610.05492* (2016).

[28] Nickolaos Koroniotis, Nour Moustafa, Elena Sitnikova, and Benjamin Turnbull. 2019. Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset. *Future Generation Computer Systems* 100 (2019), 779–796. https://doi.org/10.1016/j.future.2019.05.041

[29] Mingyan Li et al. 2025. Cognitive IoT and Edge Computing for Intrusion Detection with Federated TinyML. In *IEEE World AI IoT Congress (AIIoT)*. 677–684.

[30] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. 2020. Federated Optimization in Heterogeneous Networks. In *Proceedings of Machine Learning and Systems (MLSys)*, Vol. 2. 429–450.

[31] Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. 2020. MCUNet: Tiny Deep Learning on IoT Devices. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[32] Ji Lin, Chuang Gan, and Song Han. 2019. Defensive Quantization: When Efficiency Meets Robustness. In *International Conference on Learning Representations (ICLR)*.

[33] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Chuang Gan, and Song Han. 2022. On-Device Training Under 256KB Memory. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[34] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal Loss for Dense Object Detection. In *IEEE International Conference on Computer Vision (ICCV)*. 2980–2988.

[35] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2018. Towards Deep Learning Models Resistant to Adversarial Attacks. In *International Conference on Learning Representations (ICLR)*.

[36] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS) (PMLR, Vol. 54)*. 1273–1282.

[37] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. In *Network and Distributed System Security Symposium (NDSS)*.

[38] Nour Moustafa. 2021. A New Distributed Architecture for Evaluating AI-Based Security Systems at the Edge: Network TON_IoT Datasets. *Sustainable Cities and Society* 72 (2021), 102994. https://doi.org/10.1016/j.scs.2021.102994

[39] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. 2021. A White Paper on Neural Network Quantization. *arXiv preprint arXiv:2106.08295* (2021).

[40] Markus Nagel, Marios Fournarakis, Yelysei Bondarenko, and Tijmen Blankevoort. 2022. Overcoming Oscillations in Quantization-Aware Training. In *Proceedings of the 39th International Conference on Machine Learning (ICML) (PMLR, Vol. 162)*. 16318–16330.

[41] Markus Nagel, Mart van Baalen, Tijmen Blankevoort, and Max Welling. 2019. Data-Free Quantization Through Weight Equalization and Bias Correction. In *IEEE/CVF International Conference on Computer Vision (ICCV)*. 1325–1334.

[42] Thien Duc Nguyen, Samuel Marchal, Markus Miettinen, Hossein Fereidooni, N. Asokan, and Ahmad-Reza Sadeghi. 2019. DÏoT: A Federated Self-learning Anomaly Detection System for IoT. In *IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. 756–767.

[43] Thien Duc Nguyen, Phillip Rieger, Huili Chen, Hossein Yalame, Helen Möllering, Hossein Fereidooni, Samuel Marchal, Markus Miettinen, Azalia Mirhoseini, Shaza Zeitouni, Farinaz Koushanfar, Ahmad-Reza Sadeghi, and Thomas Schneider. 2022. FLAME: Taming Backdoors in Federated Learning. In *31st USENIX Security Symposium*.

[44] Antonio Polino, Razvan Pascanu, and Dan Alistarh. 2018. Model Compression via Distillation and Quantization. In *International Conference on Learning Representations (ICLR)*.

[45] Amirhossein Reisizadeh, Aryan Mokhtari, Hamed Hassani, Ali Jadbabaie, and Ramtin Pedarsani. 2020. FedPAQ: A Communication-Efficient Federated Learning Method with Periodic Averaging and Quantization. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS) (PMLR, Vol. 108)*. 2021–2031.

[46] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 108–116. https://doi.org/10.5220/0006639801080116

[47] Chang Song, Elias Fallon, and Hai Li. 2020. Improving Adversarial Robustness in Weight-quantized Neural Networks. *arXiv preprint arXiv:2012.14965* (2020).

[48] TensorFlow Model Optimization. 2021. Collaborative Optimization: Pruning-Preserving Quantization Aware Training. https://www.tensorflow.org/model_optimization/guide/combine/collaborative_optimization.

[49] Hao Wu, Patrick Judd, Xiaojie Zhang, Mikhail Isaev, and Paulius Micikevicius. 2020. Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation. *arXiv preprint arXiv:2004.09602* (2020).

[50] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. 2018. Federated Learning with Non-IID Data. *arXiv preprint arXiv:1806.00582* (2018).

[51] Guangmeng Zhou, Zhuotao Liu, Chuanpu Fu, Qi Li, and Ke Xu. 2023. An Efficient Design of Intelligent Network Data Plane. In *32nd USENIX Security Symposium*.

A Exact BatchNorm Folding

Each hidden layer is Dense→ReLU→BatchNorm→Dropout. At inference BatchNorm is the affine map $\mathrm{BN}(h) = s \odot h + t$ with $s = \gamma/\sqrt{\sigma^2 + \varepsilon}$ and $t = \beta - s \odot \mu$. Since it follows the ReLU it cannot be folded into the preceding Dense layer; Dropout is the identity at inference, so it is folded into the next Dense layer (W, b): $W' = \mathrm{diag}(s)\, W$, $b' = b + t^\top W$. This is exact (deviation $\leq 2 \cdot 10^{-5}$ on our models). The common pre-activation formula changed 11.6% of decisions on our layout.

B How the TON_IoT Recipe Was Chosen

Our first TON_IoT runs reused CIC-IDS2017 settings: focal $\alpha = 0.35$ down-weights the attack class, which is the majority on TON_IoT; SMOTE therefore synthesizes benign records; and the DNS query and URI fields were dropped (37 features). We trained eight variants with 10% of the training split held out and selected only on that validation set, by FAR at 99.9% validation recall (validation FAR / F1 at 0.3): initial recipe 6.84% / 99.36; $\alpha = 0.5$ 3.78% / 99.35; $\alpha = 0.65$ 7.72% / 99.06; no SMOTE 4.30% / 99.28; text-derived features 3.21% / 99.45; 100 rounds 5.18% / 99.36; text features + $\alpha = 0.5$ (selected) 3.01% / 99.39; text features + no SMOTE 3.06% / 99.41. On the test split the initial recipe misses 61 of 16,215 attacks (FAR 2.76%) and the selected one 19 (FAR 3.61%); at equal 99.9% test recall the FAR is 7.16% vs. 4.52%. *Source: run S.*

C ESP32 Benchmark Protocol

The firmware (PlatformIO, the TensorFlowLite_ESP32 1.0.0 library, the huge_app partition) embeds the four models as 16-byte-aligned arrays. For 8 fixed standardized inputs per dataset it reports the maximum output difference to the host TFLite interpreter and decision agreement at each model's threshold; after 5 warm-up runs it times 100 Invoke() calls with micros() and reports chip, clock, SDK and arena use. Builds exist for ESP32, ESP32-S3 and ESP32-C3; §4.10 uses an ESP-WROOM-32 board.

D Provenance of Reported Numbers

Every number in this paper comes from our own runs. Each run directory in the artifact holds the configuration, log, Git commit and result files (JSON/CSV/Markdown), and the artifact's PROVENANCE.md maps every table and run identifier used in the captions (A–X6) to its directory and result file. Dataset statistics (§3.1) are computed by our loaders on the published CSV files [38, 46].